

Nama : Ikhlas Putra Pambagyo
Instansi : UPN Veteran Jawa Timur
NPM : 23081010024

TUGAS LARAVEL 7

Link Repository PDF

<https://github.com/ikhlaspp/SIB-NF-Academy-Batch-4-Full-Stack-Developer>

Link Repository Project

<https://github.com/ikhlaspp/booksales-api-laravel>

1. Mengubah struktur relasi di file migration (khususnya tabel `transactions` dan tabel `books`). Kolom referensi yang sebelumnya menggunakan cara lama diubah agar mengadopsi sintaks `foreignId`, `constrained`, dan `cascadeOnDelete` atau `onDelete('cascade')`.

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('transactions', function (Blueprint $table) {
            $table->id();
            $table->string('order_number');
            $table->foreignId('customer_id')->nullable()-
>constrained('users')->onDelete('cascade');
            $table->foreignId('book_id')->nullable()-
>constrained('books')->onDelete('cascade');
            $table->decimal('total_amount', 10, 2);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('transactions');
    }
};
```

2. Mengimplementasikan pengambilan relasi data database langsung ke object Model ('Transaction' dan 'Book') menggunakan metode 'belongsTo'.

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Transaction extends Model
{
    protected $fillable = [
        'order_number',
        'customer_id',
        'book_id',
        'total_amount'
    ];

    public function customer()
    {
        return $this->belongsTo(User::class, 'customer_id');
    }

    public function book()
    {
        return $this->belongsTo(Book::class);
    }
}
```

3. Pengaturan Routing API Berdasarkan Role

```
Route::middleware(['auth:sanctum', Admin::class])->group(function () {
    Route::apiResource('genres', GenreController::class)->except(['index', 'show']);
    Route::apiResource('authors', AuthorController::class)->except(['index', 'show']);
    Route::apiResource('books', BookController::class)->except(['index', 'show']);
    Route::apiResource('transactions', TransactionController::class)->only(['index', 'destroy']);
});

Route::middleware(['auth:sanctum', Customer::class])->group(function () {
    Route::apiResource('transactions', TransactionController::class)->only(['store', 'update', 'show']);
});
```

Testing API menggunakan POSTMAN

Mensimulasikan pengguna sebagai Admin dan Customer via `POST /api/login`

The image displays two screenshots of the Postman application, demonstrating successful API login tests for an admin and a customer.

Top Screenshot (Admin Login):

- URL:** `http://localhost:8000/api/login`
- Method:** `POST`
- Body Type:** `x-www-form-urlencoded`
- Body Data:**

Key	Value	Description
email	admin@toko.com	
password	adminpass	
- Response:** `200 OK` (899 ms, 353 B). The response body is JSON:

```
{  "message": "Login berhasil",  "role": "admin",  "access_token": "1|KjAcX3uCDrG5ccBq2QvuBQtHV60nfd1TteBhtkgn70d89760",  "token_type": "Bearer"}
```

Bottom Screenshot (Customer Login):

- URL:** `http://localhost:8000/api/login`
- Method:** `POST`
- Body Type:** `x-www-form-urlencoded`
- Body Data:**

Key	Value	Description
email	budi@email.com	
password	password123	
- Response:** `200 OK` (346 ms, 356 B). The response body is JSON:

```
{  "message": "Login berhasil",  "role": "customer",  "access_token": "2|rowwyLjDomw7jknio0Gim6jRAdq8dHdpCiLAntUZ5c2daa00",  "token_type": "Bearer"}
```

Simulasi customer membeli buku

HTTP **POST** `http://localhost:8000/api/transactions` **Send**

Docs Params Authorization Headers 11 **Body** Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```
1 {
2   "order_number": "TRX-1001",
3   "book_id": 5,
4   "total_amount": 150000
5 }
```

Body Cookies 2 Headers 7 Test Results **201 Created** • 96 ms • 689 B • ...

JSON Preview Visualize

```
1 {
2   "order_number": "TRX-1001",
3   "book_id": 5,
4   "total_amount": 150000,
5   "customer_id": 1,
6   "updated_at": "2026-04-29T11:19:24.000000Z",
7   "created_at": "2026-04-29T11:19:24.000000Z",
8   "id": 11,
9   "customer": {
10    "id": 1,
11    "name": "Budi Santoso",
12    "email": "budi@email.com",
13    "role": "customer",
```

Simulasi customer melihat detail transaksi

HTTP **GET** `http://localhost:8000/api/transactions/1` **Send**

Docs Params Authorization Headers 9 **Body** Scripts Tests Settings Cookies

Body Cookies 2 Headers 7 Test Results **200 OK** • 98 ms • 682 B • ...

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "order_number": "ORD-001",
4   "customer_id": 1,
5   "book_id": 1,
6   "total_amount": "75000.00",
7   "created_at": "2026-04-29T11:18:46.000000Z",
8   "updated_at": "2026-04-29T11:18:46.000000Z",
9   "customer": {
10    "id": 1,
11    "name": "Budi Santoso",
12    "email": "budi@email.com",
13    "role": "customer",
14    "last_access": "2026-04-29 11:18:46"
15  },
16  "book": {
17    "id": 1,
18    "title": "Laskar Pelangi",
19    "description": "Kisah 10 anak di Belitung.",
20    "price": "80000.00",
21    "stock": 29,
```

Simulasi customer mengupdate transaksi

The screenshot shows a REST client interface with a PUT request to `http://localhost:8000/api/transactions/1`. The request body is a JSON object with `book_id` and `total_amount`. The response is a 200 OK status with a JSON object containing transaction details and customer information.

```
PUT http://localhost:8000/api/transactions/1
```

```
{
  "book_id": 3,
  "total_amount": 190000
}
```

200 OK • 103 ms • 691 B

```
{
  "id": 1,
  "order_number": "ORD-001",
  "customer_id": 1,
  "book_id": 3,
  "total_amount": 190000,
  "created_at": "2026-04-29T11:18:46.000000Z",
  "updated_at": "2026-04-29T11:20:56.000000Z",
  "customer": {
    "id": 1,
    "name": "Budi Santoso",
    "email": "budi@email.com",
    "role": "customer",
    "last_access": "2026-04-29 11:18:46"
  }
}
```

Simulasi admin melihat seluruh data transaksi (show all)

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/transactions`. The response is a 200 OK status with a JSON array containing transaction details and book information.

```
GET http://localhost:8000/api/transactions
```

200 OK • 82 ms • 5.27 KB

```
[
  {
    "id": 1,
    "order_number": "ORD-001",
    "customer_id": 1,
    "book_id": 3,
    "total_amount": "190000.00",
    "created_at": "2026-04-29T11:18:46.000000Z",
    "updated_at": "2026-04-29T11:20:56.000000Z",
    "customer": {
      "id": 1,
      "name": "Budi Santoso",
      "email": "budi@email.com",
      "role": "customer",
      "last_access": "2026-04-29 11:18:46"
    },
    "book": {
      "id": 3,
      "title": "Harry Potter dan Batu Bertuah",
      "description": "Awal kisah penyihir cilik.",
      "price": "150000.00",
    }
  }
]
```

Simulasi admin menghapus salah satu data transaksi

